

Attribute-Based Encryption

Attribute-Based Encryption (ABE) Access Control System for Digital Agencies

Ayşe İrem Çolak

Purpose

The goal is to cryptographically encrypt sensitive files within companies (such as design and code files) based on the titles, departments, and authorization periods of the individuals involved. This developed smart contract system is a digital lock engine that allows only those with the correct qualifications and current authorization to decrypt and access the file.

Reference Project Analysis and Comparison

- A Python-based ABAC (Attribute-Based Access Control) simulator with multiple fields was used as a reference.
- In the reference project, user requests and access rules were read and processed from text files by a central Python script (myabac.py), and PERMIT/DENY outputs were returned to the terminal.
- Based on this work, I took this access logic, which revolves around temporary and centralized memory, and transformed it into an immutable, cryptographically locked, and decentralized blockchain architecture by writing it from scratch in Solidity.

Requirements of the Project

- **Implement attribute validation system:** The ``provisionUserAttributes`` function fulfills this requirement. It dynamically verifies the set of user attributes defined by the Central Security Authority and the blockchain network's current timestamp.
- **Encrypt/decrypt based on policy:** The developed ``lockAssetWithPolicy`` function, rather than storing data in the system as plain text, links it to a specific access policy and cryptographically secures it by hashing the password using the Keccak-256 algorithm.
- **Support multi-attribute access control:** The ``evaluateAndDecrypt`` function not only verifies the user's single role; it also uses a ``for`` loop to verify multiple attributes—such as the department, hierarchy, and time validity—stored in the wallet at the same time (multi-attribute verification).
- **Privacy Concept(Fine-grained access control):** The 'Fine-grained' concept has been successfully implemented through this three-tiered control layer. Even if the user is a senior designer, the system blocks access at the micro level if the timestamp has expired or if the user does not have the cryptographic key associated with the specific resource.

Test Scenarios

To test the system's fault tolerance, an automated test scenario consisting of 5 different input matrices has been configured in the Hardhat environment. When the 'npx hardhat test' command is run, the system produces the following output:

- Test Input 1: Should **GRANT** access for valid attributes matching the ciphertext policy.
- Test Input 2: Should **DENY** access if the user's attribute key lifetime has expired.
- Test Input 3: Should **DENY** access if the user's department attribute doesn't match the policy.
- Test Input 4: Should **DENY** access if attributes match but the password is wrong.
- Test Input 5: Should **DENY** access if the user address holds no attributes in the registry.

- REFERENCE PROJECT OUTPUT

```
ayseiremcolak@Ayse-MacBook-Air ABAC-Simulator % python3 myabac.py university.abac university-requests.txt
csStu1,cs101gradebook,addScore: DENY
csStu1,cs101gradebook,readMyScores: PERMIT
csStu2,cs601gradebook,addScore: DENY
csStu2,cs602gradebook,addScore: DENY
csStu3,cs602gradebook,readMyScores: PERMIT
```

- ORIGINAL PROJECT OUTPUT

AgencyABEControl – Comprehensive 5-Input Evaluation Matrix

- ✓ Test Input 1: Should GRANT access for valid attributes matching the ciphertext policy
- ✓ Test Input 2: Should DENY access if the user's attribute key lifetime has expired
- ✓ Test Input 3: Should DENY access if the user's department attribute doesn't match the policy
- ✓ Test Input 4: Should DENY access if attributes match but the secret token is wrong
- ✓ Test Input 5: Should DENY access if the user address holds no attributes in the registry

5 passing (276ms)

Live Interactive Testing

PHASE 1: Dynamic On-Chain Asset Locking (Encryption)

- Enter a unique Asset ID: Provide any unassigned numerical identifier (e.g., 202 or 999).
- Set the department policy constraint: Define the mandatory access string required to unlock this specific asset (e.g., finans or tasarim).
- Set the cryptographic passphrase: Define a raw secret passphrase string. The deployment framework will automatically pass this through Keccak-256 hashing structures on-chain.
- Enter secure resource pointer/URI: Provide a mock decentralized pointer path (e.g., ipfs://my-private-ledger).

```
=====
🔒 PHASE 1: DYNAMIC ON-CHAIN ASSET LOCKING (ENCRYPTION)
=====
Enter a unique Asset ID to lock (e.g., 202 or 999): 202
Set the department policy constraint for this asset (e.g., 'tasarim' or 'finans'): finans
Set the cryptographic passphrase for this asset: irem_secret_key
Enter the secure resource pointer/URI (e.g., ipfs://my-private-data): ipfs://my-private-data

[Cryptographic Minter]: Hashing the passphrase via Keccak-256...
[EVM State Transition]: Executing lockAssetWithPolicy for Asset ID 202...
🟢 SUCCESS: Asset 202 is now cryptographically locked on-chain under the 'finans' policy!
```

PHASE 2: Dynamic User Provisioning & Decryption Verification

- Assign department attribute token: Input the department tag for the test worker. (To simulate a successful match, input the exact string you defined in Phase 1. To trigger a policy breach defense, input a mismatching tag like pazarlama).
- Set token lifetime status: Type yes to grant an active, valid Unix time-lock window, or no to instantly seed an expired epoch.
- Provide cryptographic passphrase token: Type the passphrase to attempt decryption.

```
=====
🔑 PHASE 2: DYNAMIC USER PROVISIONING & DECRYPTION
=====
1. Assign a department attribute token to the user (To PASS use 'finans', to FAIL use something else): finans
2. Set attribute token lifetime status. Should it be valid? (yes / no): yes

[Ledger Report]: Dynamic attributes successfully mined on-chain for 'Juri_User'.
3. Provide the cryptographic passphrase token to unlock Asset 202: irem_secret_key

[Cryptographic Processing]: Contract evaluating state transitions on the EVM...

===== LIVE EVALUATION VERDICT =====
🟢 DECRYPTION GRANTED (ALLOWED: TRUE)
📁 Unlocked IPFS Secure Varlık Pointer: DECRYPTION GRANTED. Payload: ipfs://my-private-data
=====
```